Universidad de las Fuerzas Armadas ESPE

Carrera de Ingeniería de Software

Desarrollo de Aplicaciones Móviles

Autores

Luciana Francheska Bolaños Toapanta

Daniela Alejandra Flores Toapanta

Joffre Esteban Gomez Quinaluisa

Marco Adrian Padilla Triviño

Grupo 5 *NRC: 36873*

GitHub: github.com/Adrizzx/aplicaciones-moviles-g5

Tema: Creación y ejecución de una Activity

Tarea / Resolución de Problemas 1.1

Ing. Doris Karina Chicaiza Angamarca

Departamento de Ciencias de la Computación - ESPE

Índice

1. Introducción **3**

2. Objetivos **3**

2.1. Objetivo general 3

2.2. Objetivos específicos 3

3. Trabajos relacionados **4**

4. Desarrollo **5**

4.1. Conceptos básicos 5

4.2. Herramientas 5

4.2.1. Flutter y Dart 5

4.2.2. Visual Studio Code 6

4.2.3. Android Studio y emulador 6

4.2.4. GitHub 6

4.3. Desarrollo paso a paso 6

4.4. Pruebas 20

5. Conclusiones **22**

6. Recomendaciones **22**

7. Bibliografía **23**

1. Introducción

En Android, una *Activity* representa una pantalla con la que el usuario interactúa. En Flutter esa idea se traslada a los *widgets* y a las rutas: cada pantalla es un widget que se apila o se retira mediante el `Navigator` (Google, 2026d). Entender cómo se crea una pantalla, cómo se llega a ella y cómo se regresa es uno de los primeros pasos para desarrollar aplicaciones móviles con este framework.

En esta tarea, el Grupo 5 del NRC 36873 desarrolló una única aplicación en Flutter y Dart que resuelve los ejercicios 6 al 10 asignados a su NRC: impuesto de vehículos, saldos de un banco, ahorro anual, un juego para adivinar un número y el análisis de números dentro de un intervalo. La aplicación tiene un menú principal desde el cual se abre cada ejercicio en una pantalla independiente, usando rutas con nombre, y desde cada pantalla se puede volver al menú. El código se organizó separando el punto de entrada (`main.dart`), la lógica de los ejercicios y las pantallas, como pide la consigna. Además, cada formulario valida los datos ingresados y la lógica fue comprobada con pruebas automáticas.

2. Objetivos

2.1. Objetivo general

Desarrollar una aplicación móvil en Flutter y Dart que resuelva los ejercicios 6 al 10, mediante un menú principal enlazado a pantallas independientes con el sistema de rutas y navegación de Flutter, para practicar la creación, ejecución y navegación entre pantallas de una aplicación móvil.

2.2. Objetivos específicos

- Implementar la navegación entre el menú principal y las cinco pantallas de ejercicios utilizando rutas con nombre y el retorno al menú.
- Organizar el proyecto separando el archivo principal, la lógica de cada ejercicio, las pantallas, las validaciones y los widgets reutilizables.
- Validar las entradas de cada ejercicio y comprobar el funcionamiento de la lógica con pruebas unitarias y con la ejecución de la aplicación en un emulador Android.

3. Trabajos relacionados

El desarrollo de aplicaciones con frameworks multiplataforma como Flutter ha sido analizado desde distintos puntos de vista. Los siguientes cinco trabajos sirven como base para la elección de la herramienta y la forma de trabajo de esta tarea.

1. **Conceptos y desafíos del desarrollo multiplataforma.** Biørn-Hansen et al. (2018) revisaron la literatura sobre desarrollo móvil multiplataforma y propusieron una taxonomía de enfoques. El trabajo permite ubicar a Flutter dentro de los frameworks que compilan a código nativo y explica por qué una sola base de código puede generar aplicaciones para varias plataformas, como ocurrió en esta tarea con Android, Windows y web.

2. **Rendimiento de frameworks multiplataforma en Android.** Mahendra y Anggorojati (2020) midieron el rendimiento de aplicaciones Android construidas con distintos frameworks multiplataforma. Sus resultados muestran que la elección del framework influye en el consumo de recursos y en la fluidez, un aspecto a considerar al ejecutar la aplicación en el emulador.

3. **Experiencia práctica con Flutter.** Kuzmin et al. (2020) relataron el desarrollo de una aplicación real con Flutter para Android e iOS desde un único código. Destacan la rapidez para construir interfaces con widgets, que es justamente la forma en que se armaron el menú y las pantallas de esta aplicación.

4. **Comparación entre frameworks nativos y multiplataforma.** Nawrocki et al. (2021) compararon frameworks nativos y multiplataforma, incluido Flutter, en aplicaciones equivalentes. Concluyen que Flutter ofrece un desempeño cercano al nativo, lo que respalda su uso en la asignatura.

5. **Análisis empírico de Kotlin, React Native y Flutter.** Suri et al. (2022) evaluaron de forma empírica aplicaciones desarrolladas con Kotlin, React Native y Flutter. El estudio resalta la productividad de Flutter y su enfoque basado en widgets, en relación directa con la organización por pantallas usada en este trabajo.

4. Desarrollo

4.1. Conceptos básicos

- **Activity y pantalla en Flutter:** en Android nativo una Activity es una pantalla de la aplicación. En Flutter cada pantalla es un widget, normalmente un `Scaffold` con su barra superior y su contenido.
- **Widget sin estado y con estado:** un `StatelessWidget` solo muestra información, como el menú principal. Un `StatefulWidget` cambia lo que muestra cuando el usuario interactúa, como las pantallas de los ejercicios, que se actualizan con `setState`.
- **Navigator y rutas:** el `Navigator` maneja las pantallas como una pila. `push` coloca una pantalla nueva encima y `pop` la retira para volver a la anterior (Google, 2026d).
- **Rutas con nombre:** cada pantalla se registra en el mapa `routes` de `MaterialApp` con un texto como `'/ejercicio6'`, y se abre con `Navigator.pushNamed`. La documentación indica que esta forma es adecuada para aplicaciones sencillas como la de esta tarea (Google, 2026c).
- **Formularios y validación:** un `Form` con una `GlobalKey` agrupa varios `TextFormField`; cada campo tiene una función `validator` que devuelve un mensaje de error o `null` si el dato es correcto (Google, 2026a).
- **Pruebas unitarias:** verifican el comportamiento de una función o clase de forma aislada. En Flutter se escriben en la carpeta `test` y se ejecutan con `flutter test` (Google, 2026b).

4.2. Herramientas

4.2.1. Flutter y Dart

Se utilizó Flutter 3.47.6 del canal *stable* con Dart 3.13.5, la misma instalación configurada en el Laboratorio 1.1. Flutter proporciona los widgets de Material Design y el sistema de navegación; Dart es el lenguaje en el que se escribió toda la aplicación.

4.2.2. Visual Studio Code

Se usó como editor principal, con las extensiones de Flutter y Dart. Desde su terminal integrada se ejecutaron los comandos de creación, análisis, pruebas y ejecución del proyecto.

4.2.3. Android Studio y emulador

Android Studio se empleó para administrar el SDK de Android y el emulador. La aplicación se ejecutó en un dispositivo virtual Pixel 7 con Android 16 (API 36), identificado como `Pixel_7_API_36`.

4.2.4. GitHub

Se utilizó GitHub, junto con GitHub Desktop, para alojar el código fuente y el informe en un repositorio compartido por los integrantes del grupo.

4.3. Desarrollo paso a paso

Paso 1: Creación del proyecto y estructura

Se creó el proyecto `pry_tarea1_1_g5` y se generaron las carpetas de las plataformas Android, Windows y web con el siguiente comando, ejecutado dentro de la carpeta del proyecto:

```
# terminal
flutter create . --org ec.edu.espe --platforms=android,windows,
    web
flutter pub get
```

El código fuente completo de la aplicación y este informe se encuentran en el repositorio del grupo, dentro de la carpeta `Tarea1.1`:

https://github.com/Adrizzx/aplicaciones-moviles-g5

El código se organizó en carpetas dentro de `lib`, separando cada responsabilidad (Figura 1):

- `main.dart`: punto de entrada, tema de la aplicación y tabla de rutas.
- `rutas.dart`: nombres de las rutas en un solo lugar.

- `logic/`: un archivo por ejercicio con los cálculos, escritos en Dart puro y sin depender de la interfaz.
- `screens/`: el menú y las cinco pantallas de los ejercicios.
- `utils/`: funciones de validación y de formato de números.
- `widgets/`: componentes que se repiten, como el botón para volver al menú y la tarjeta de resultados.

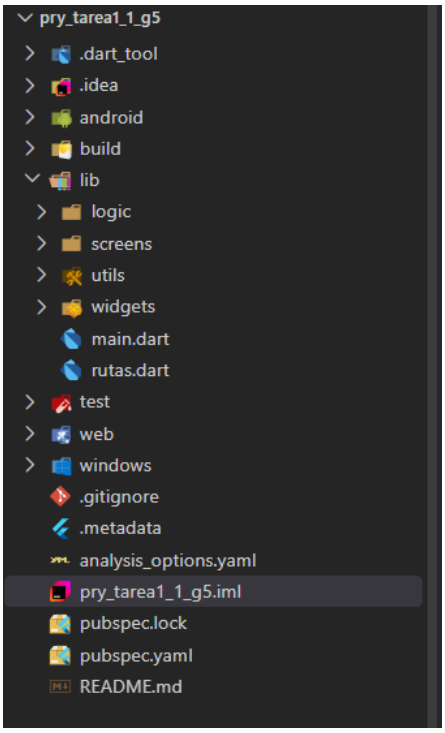


Figura 1: Estructura del proyecto con las carpetas logic, screens, utils y widgets dentro de lib.

Paso 2: Rutas y archivo principal

Primero se definieron los nombres de las rutas en una clase aparte, para no repetir textos en distintos archivos:

```
1 // rutas.dart
2 class Rutas {
3   static const String menu = '/';
```

```
  static const String ejercicio6 = '/ejercicio6';
  static const String ejercicio7 = '/ejercicio7';
  static const String ejercicio8 = '/ejercicio8';
  static const String ejercicio9 = '/ejercicio9';
  static const String ejercicio10 = '/ejercicio10';
}
```

En `main.dart` se registraron esas rutas en el `MaterialApp`. La propiedad `initialRoute` indica que la aplicación arranca en el menú, y el mapa `routes` asocia cada nombre con su pantalla:

```
// main.dart
void main() {
  runApp(const MiApp());
}

class MiApp extends StatelessWidget {
  const MiApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Tarea 1.1 - Grupo 5',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal)
            ,
      ),
      initialRoute: Rutas.menu,
      routes: {
        Rutas.menu: (context) => const MenuScreen(),
        Rutas.ejercicio6: (context) => const Ejercicio6Screen(),
        Rutas.ejercicio7: (context) => const Ejercicio7Screen(),
        Rutas.ejercicio8: (context) => const Ejercicio8Screen(),
        Rutas.ejercicio9: (context) => const Ejercicio9Screen(),
        Rutas.ejercicio10: (context) => const Ejercicio10Screen()
            ,
      },
```

```
    );
  }
}
```

Paso 3: Menú principal y navegación

El menú es un `StatelessWidget` con una lista de tarjetas, una por ejercicio. Al tocar una tarjeta se llama a `Navigator.pushNamed` con el nombre de la ruta, lo que coloca la pantalla del ejercicio encima del menú (Figura 2).

```dart
// menu_screen.dart (fragmento)
for (final opcion in _opciones)
  Card(
    child: ListTile(
      leading: Icon(opcion.icono),
      title: Text(opcion.titulo),
      subtitle: Text(opcion.descripcion),
      trailing: const Icon(Icons.arrow_forward_ios, size: 16),
      onTap: () => Navigator.pushNamed(context, opcion.ruta),
    ),
  ),
```

Para regresar, cada pantalla tiene la flecha de la barra superior, que Flutter agrega sola, y un botón *Volver al menú* que retira la pantalla de la pila con `Navigator.pop` (Figura 3):

```dart
// comunes.dart (fragmento)
class BotonVolver extends StatelessWidget {
  const BotonVolver({super.key});

  @override
  Widget build(BuildContext context) {
    return OutlinedButton.icon(
      onPressed: () => Navigator.pop(context),
      icon: const Icon(Icons.home),
      label: const Text('Volver al menú'),
    );
  }
}
```

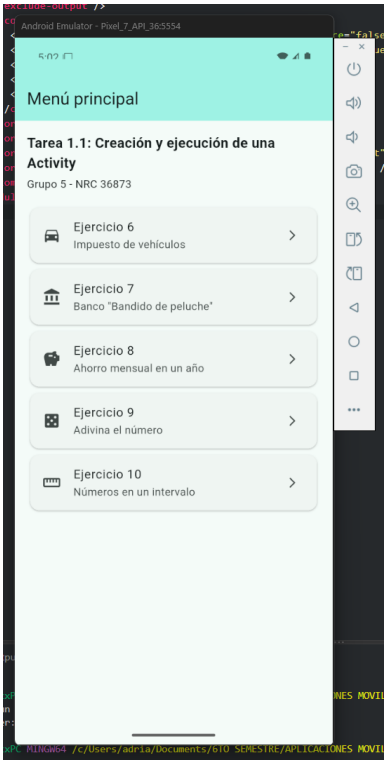


Figura 2: Menú principal de la aplicación con los ejercicios 6 al 10, ejecutado en el emulador Pixel 7.



Figura 3: Botón Volver al menú presente al final de cada pantalla de ejercicio.

Paso 4: Validaciones comunes

Para no repetir código, las validaciones se agruparon en `validaciones.dart`. La función `leerDecimal` acepta tanto punto como coma decimal y descarta textos que no son números; las funciones `validar...` se usan directamente como `validator` de los campos del formulario:

```
1 // validaciones.dart (fragmento)
2 double? leerDecimal(String? texto) {
3   if (texto == null) return null;
```

```
  final limpio = texto.trim().replaceAll(',', '.');
  if (limpio.isEmpty) return null;
  final valor = double.tryParse(limpio);
  if (valor == null || !valor.isFinite) return null;
  return valor;
}

String? validarNoNegativo(String? texto) {
  final error = validarNumero(texto);
  if (error != null) return error;
  if (leerDecimal(texto)! < 0) {
    return 'No se permiten valores negativos';
  }
  return null;
}
```

Paso 5: Ejercicio 6, impuesto de vehículos

Para cada vehículo se elige su clave en una lista desplegable y se escribe su valor. El impuesto depende de la clave: 10 % para la clave 1, 7 % para la clave 2 y 5 % para la clave 3. La aplicación muestra el impuesto de cada vehículo, el total por categoría y el total general.

```
// ejercicio6_logica.dart (fragmento)
static double porcentaje(int clave) {
  switch (clave) {
    case 1:
      return 0.10;
    case 2:
      return 0.07;
    case 3:
      return 0.05;
    default:
      throw ArgumentError('La clave debe ser 1, 2 o 3');
  }
}

double totalPorClave(int clave) {
```

```
16    double total = 0;
17    for (final v in vehiculos) {
18      if (v.clave == clave) total += v.impuesto;
19    }
20    return total;
21  }
```

Si el valor se deja vacío, es negativo o es cero, el formulario no deja agregar el vehículo (Figura 4). Con los datos de prueba, cuatro vehículos de valores \$10 000 (clave 1), \$20 000 (clave 2), \$30 000 (clave 3) y \$5 000 (clave 1), se obtuvo un total de \$1 500 para la clave 1, \$1 400 para la clave 2, \$1 500 para la clave 3 y \$4 400 en total (Figura 5).

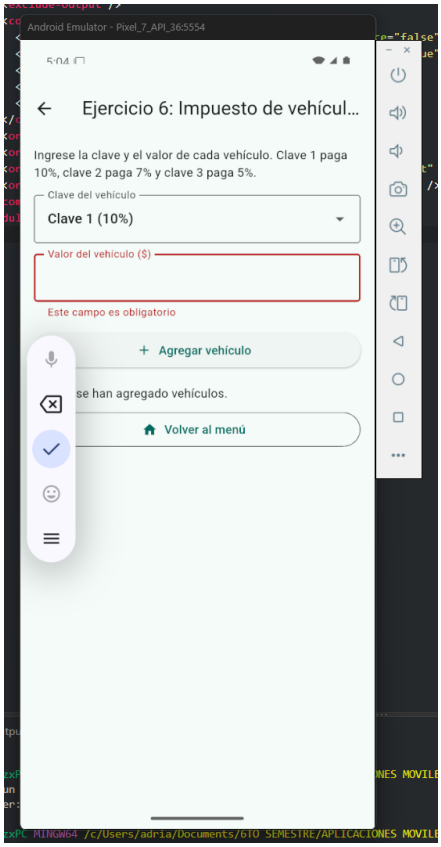


Figura 4: Ejercicio 6: validación del valor vacío del vehículo.

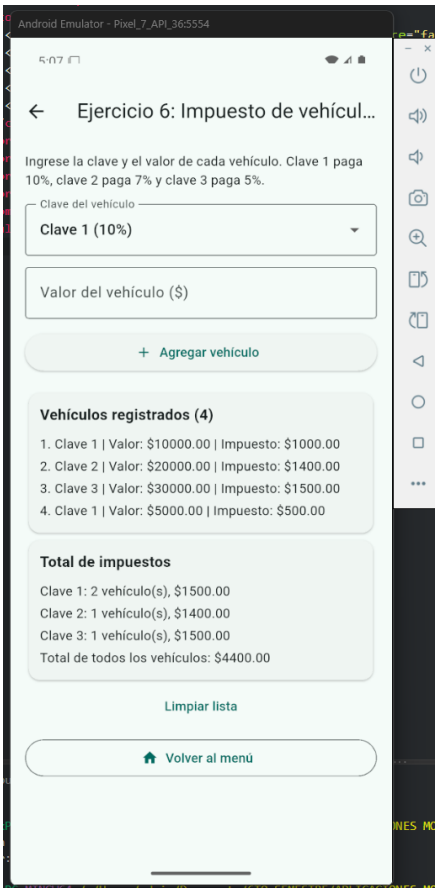


Figura 5: Ejercicio 6: impuestos por clave y total de \$4400.00.

Paso 6: Ejercicio 7, banco “Bandido de peluche”

Para cada cliente se ingresan el nombre, el saldo anterior, el monto de compras y el pago realizado en el corte anterior. Se consideró moroso al cliente que pagó menos del 15 % de su saldo anterior; en ese caso se le cobra 12 % de interés sobre lo que quedó debiendo y una multa de $200. Con eso se obtiene el saldo actual, el pago mínimo (15 % del saldo actual) y el pago para no generar intereses (85 % del saldo actual).

```
// ejercicio7_logica.dart (fragmento)
pagoMinimoAnterior = saldoAnterior * Ejercicio7.
    porcentajePagoMinimo;
esMoroso = pagoRealizado < pagoMinimoAnterior;

if (esMoroso) {
  intereses = (saldoAnterior - pagoRealizado) * Ejercicio7.
      porcentajeInteres;
  multa = Ejercicio7.multa;
} else {
  intereses = 0;
  multa = 0;
}

saldoActual = saldoAnterior - pagoRealizado + compras + intereses
    + multa;
pagoMinimo = saldoActual * Ejercicio7.porcentajePagoMinimo;
pagoSinIntereses = saldoActual * Ejercicio7.
    porcentajeSinIntereses;
```

Además de los campos obligatorios y no negativos, la lógica rechaza un pago mayor que el saldo anterior más las compras, y la pantalla lo informa con un mensaje (Figura 6). En la prueba, Ana (saldo $1 000, compras $500, pago $200) quedó al día con un saldo de $1 300. Luis (saldo $1 000, compras $300, pago $100) no cubrió el mínimo de $150, por lo que se le sumaron $108 de intereses y $200 de multa, con un saldo final de $1 508. El resumen del banco muestra una ganancia de $108 por intereses (Figura 7).

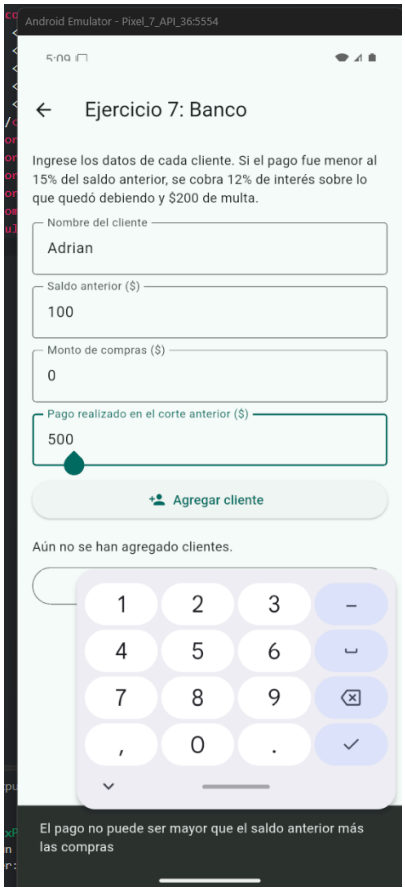


Figura 6: Ejercicio 7: aviso cuando el pago supera el saldo anterior más las compras.

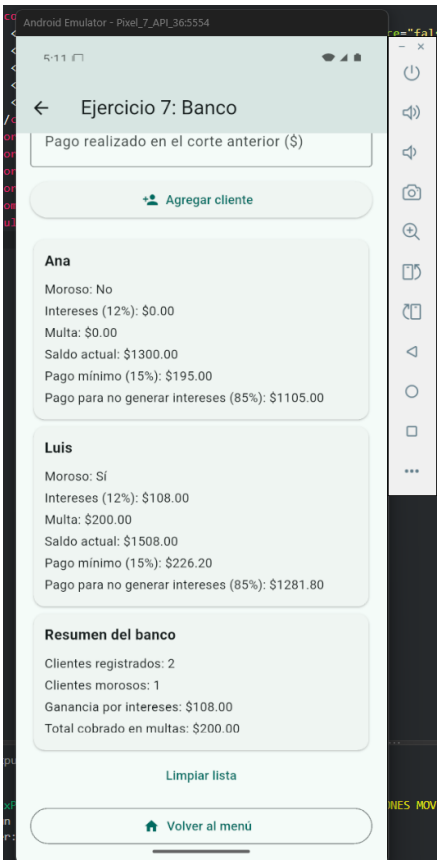


Figura 7: Ejercicio 7: resultados de los clientes y resumen del banco.

Paso 7: Ejercicio 8, ahorro anual

La pantalla presenta doce campos, uno por mes. Al presionar *Calcular* se suma cada depósito al total acumulado y se muestra cuánto se lleva ahorrado al final de cada mes, además del total del año.

```
// ejercicio8_logica.dart (fragmento)
static List<double> calcularAcumulado(List<double> depositos) {
  if (depositos.length != 12) {
    throw ArgumentError('Se necesitan los depósitos de los 12
        meses');
  }
  final acumulado = <double>[];
```

```
7    double suma = 0;
8    for (final deposito in depositos) {
9      if (deposito < 0) {
10       throw ArgumentError('Los depósitos no pueden ser negativos'
             );
11     }
12     suma += deposito;
13     acumulado.add(suma);
14   }
15   return acumulado;
16 }
```

Todos los meses son obligatorios; se permite 0 para los meses sin ahorro, pero no valores vacíos ni negativos (Figura 8). Con depósitos de \$10, \$20, ..., \$120 el total anual fue de \$780 (Figura 9).

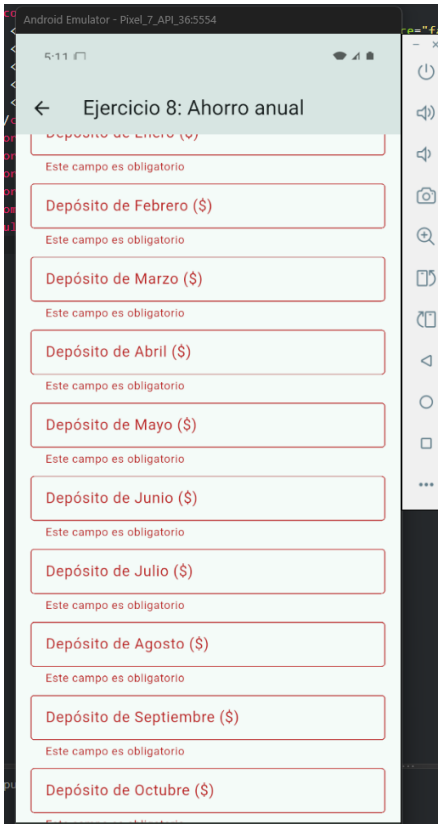


Figura 8: Ejercicio 8: validación de los meses sin depósito ingresado.

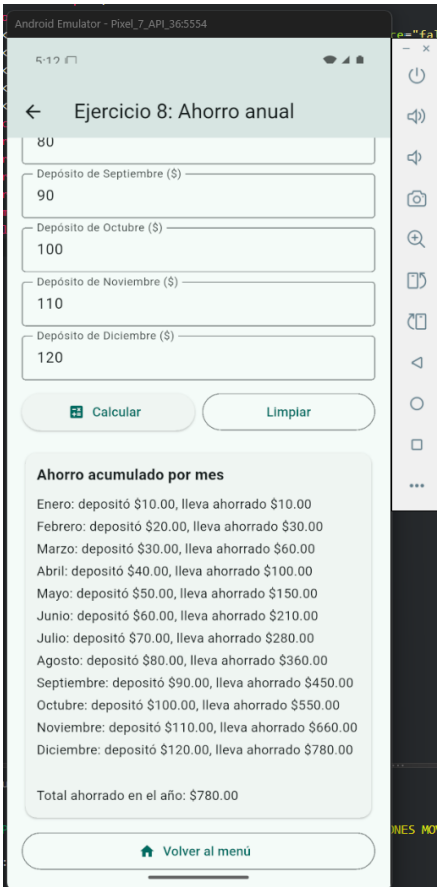


Figura 9: Ejercicio 8: ahorro acumulado por mes y total anual de $780.00.

Paso 8: Ejercicio 9, adivina el número

Al abrir la pantalla se genera un número aleatorio entre 1 y 100 con la clase `Random`. El usuario tiene 10 intentos; después de cada uno la aplicación indica si el número secreto es mayor o menor y cuántos intentos quedan. El juego termina al acertar, mostrando en cuántos intentos se logró, o al agotar los intentos, mostrando el número generado.

```dart
// ejercicio9_logica.dart (fragmento)
ResultadoIntento intentar(int numero) {
  if (terminado) {
    throw StateError('El juego ya terminó');
  }
  if (numero < minimo || numero > maximo) {
```

```
7       throw ArgumentError('El número debe estar entre $minimo y
            $maximo');
8     }
9
10    intentosUsados++;
11
12    Pista pista;
13    if (numero == numeroSecreto) {
14      gano = true;
15      pista = Pista.acierto;
16    } else if (numeroSecreto > numero) {
17      pista = Pista.mayor;
18    } else {
19      pista = Pista.menor;
20    }
21    return ResultadoIntento(numero, pista, intentosRestantes);
22  }
```

El campo solo acepta números enteros entre 1 y 100. En la Figura 10 se observan los primeros intentos con sus pistas; en la Figura 11 el número 71 se acertó en el noveno intento y aparece la opción para jugar otra vez.

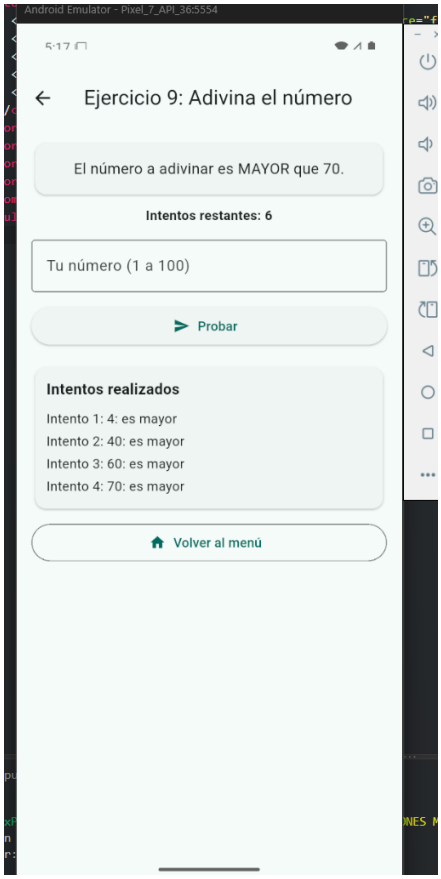


Figura 10: Ejercicio 9: pistas e intentos restantes durante el juego.

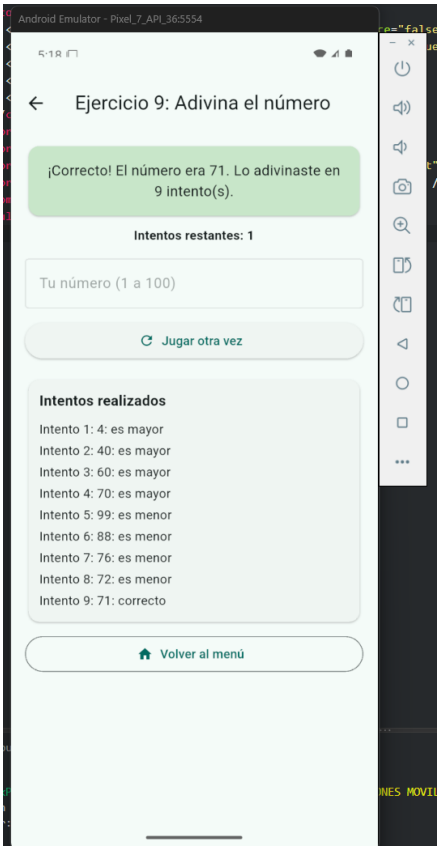


Figura 11: Ejercicio 9: número acertado en 9 intentos.

Paso 9: Ejercicio 10, números en un intervalo

Primero se ingresan los límites del intervalo. Si el límite inferior es mayor o igual que el superior, el campo marca el error y se vuelven a pedir los datos. Luego se ingresan números uno por uno hasta escribir 0. Al terminar se muestra la suma de los números que están dentro del intervalo abierto, la cantidad de números que quedaron fuera y si se ingresó alguno igual a los límites.

```dart
// ejercicio10_logica.dart (fragmento)
void agregar(double numero) {
  if (terminado) {
    throw StateError('Ya se ingresó el 0, el ingreso terminó');
  }
```

```
6     if (numero == 0) {
7       terminado = true;
8       return;
9     }
10
11    numeros.add(numero);
12
13    if (numero > inferior && numero < superior) {
14      sumaDentro += numero;
15    } else if (numero == inferior || numero == superior) {
16      hayIgualALimite = true;
17    } else {
18      cantidadFuera++;
19    }
20  }
```

Como el intervalo es abierto, un número igual a un límite no se suma ni se cuenta como fuera; solo se informa aparte, como pide el enunciado. Con el intervalo $(10, 20)$ y los números $15, 12, 25, 5, 10$ y 0, la suma fue 27, quedaron 2 números fuera y se detectó un número igual al límite (Figura 13).

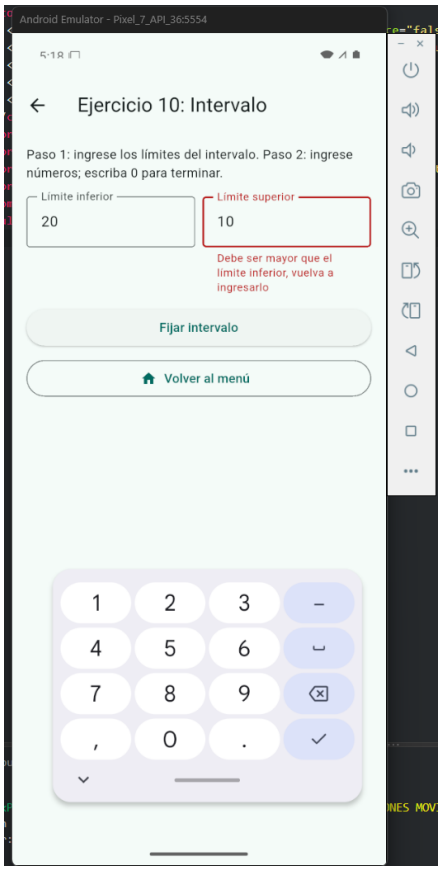


Figura 12: Ejercicio 10: límite inferior mayor que el superior, se vuelve a pedir.

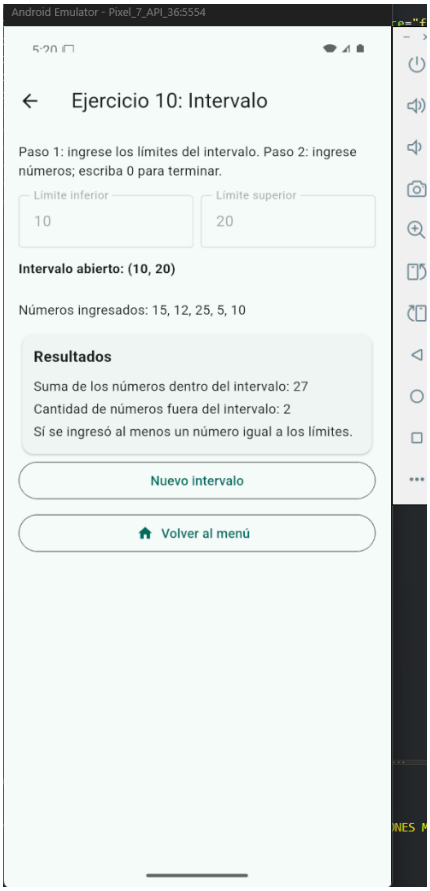


Figura 13: Ejercicio 10: resultados con el intervalo (10, 20).

4.4. Pruebas

Además de probar cada pantalla de forma manual en el emulador, se ejecutaron tres verificaciones desde la terminal de Visual Studio Code, dentro de la carpeta del proyecto:

```
# terminal
flutter analyze
flutter test
flutter run -d emulator-5554
```

- **Análisis estático:** `flutter analyze` revisa el código con las reglas de `flutter_lints`. El resultado fue *No issues found!*, es decir, sin errores ni advertencias.

- **Pruebas automáticas:** `flutter test` ejecutó las 17 pruebas del proyecto y todas pasaron (*All tests passed!*). Su detalle está en la Tabla 1.
- **Ejecución:** `flutter run` compiló la aplicación con Gradle y la instaló en el emulador Pixel 7 con Android 16 (API 36), donde se tomaron las capturas de este informe. La aplicación también se ejecutó como aplicación de escritorio de Windows con `flutter run -d windows`.

Grupo	Qué se comprueba
Validaciones (2)	Lectura de números con punto o coma; rechazo de campos vacíos, letras y negativos.
Ejercicio 6 (2)	Impuesto por clave y totales con los datos de prueba; rechazo de clave inválida y valor menor o igual a 0.
Ejercicio 7 (3)	Cliente al día sin recargos; cliente moroso con 12 % de interés y $200 de multa; rechazo de datos inválidos.
Ejercicio 8 (2)	Acumulado de cada mes y total de $780; rechazo de menos de 12 meses o depósitos negativos.
Ejercicio 9 (4)	Pistas y fin del juego al acertar; fin al agotar los 10 intentos; rechazo de números fuera de 1 a 100; número aleatorio siempre entre 1 y 100.
Ejercicio 10 (3)	Suma, números fuera y detección de límites; rechazo de límites invertidos; bloqueo de datos después del 0.
Navegación (1)	El menú muestra los cinco ejercicios, abre el ejercicio 9 y regresa al menú con la flecha.

Tabla 1: Pruebas automáticas ejecutadas con flutter test (17 en total).

Un ejemplo de las pruebas unitarias es el caso del cliente moroso del Ejercicio 7, donde se comparan los valores calculados con los obtenidos a mano:

```
1 // logica_test.dart (fragmento)
2 test('cliente moroso: 12% de interés y \$200 de multa', () {
3   final e = Ejercicio7();
4   // Pago mínimo anterior = 150, pagó 100 -> moroso
5   final c = e.agregarCliente(
6       nombre: 'Luis', saldoAnterior: 1000, compras: 300,
          pagoRealizado: 100);
7
8   expect(c.esMoroso, isTrue);
```

```
9     expect(c.intereses, closeTo(108, 0.001)); // 12% de 900
10    expect(c.multa, 200);
11    expect(c.saldoActual, closeTo(1508, 0.001)); // 900 + 300 + 108
          + 200
12  });
```

5. Conclusiones

- El sistema de rutas con nombre permitió enlazar el menú principal con las cinco pantallas de forma sencilla: cada ruta se registra una sola vez en `main.dart` y se abre con `pushNamed`, mientras que `pop` devuelve al usuario al menú. Para una aplicación de este tamaño, este enfoque resultó suficiente y fácil de mantener.
- Separar la lógica de los ejercicios de las pantallas facilitó tanto la lectura del código como su verificación, ya que los cálculos pudieron probarse con pruebas unitarias sin abrir la interfaz. Las 17 pruebas aprobadas y el análisis sin observaciones confirman que los resultados mostrados coinciden con los cálculos esperados.
- Validar cada entrada mediante `Form` y `TextFormField` evitó que la aplicación procese datos vacíos, negativos o fuera de rango, y le indica al usuario qué corregir. Esto fue especialmente importante en los ejercicios 9 y 10, donde un dato inválido podía consumir un intento o alterar los resultados.

6. Recomendaciones

- Para aplicaciones más grandes o con soporte web, migrar de rutas con nombre a un enrutador declarativo como `go_router`, ya que la documentación de Flutter señala limitaciones de las rutas con nombre en enlaces profundos y en el navegador.
- Escribir las pruebas unitarias junto con la lógica de cada ejercicio y ejecutar `flutter test` antes de cada entrega, para detectar a tiempo errores en los cálculos.
- Definir con el docente las interpretaciones de los enunciados ambiguos antes de programar, como el cálculo de intereses del Ejercicio 7, y dejarlas documentadas en el informe y en el código.

7. Bibliografía

Biørn-Hansen, A., Grønli, T.-M., & Ghinea, G. (2018). A Survey and Taxonomy of Core Concepts and Research Challenges in Cross-Platform Mobile Development. *ACM Computing Surveys*, *51*(5), 1-34. https://doi.org/10.1145/3241739

Google. (2026a). *Build a form with validation*. Consultado el 12 de octubre de 2026, desde https://docs.flutter.dev/cookbook/forms/validation

Google. (2026b). *An introduction to unit testing*. Consultado el 12 de octubre de 2026, desde https://docs.flutter.dev/cookbook/testing/unit/introduction

Google. (2026c). *Navigate with named routes*. Consultado el 12 de octubre de 2026, desde https://docs.flutter.dev/cookbook/navigation/named-routes

Google. (2026d). *Navigation and routing*. Consultado el 12 de octubre de 2026, desde https://docs.flutter.dev/ui/navigation

Kuzmin, N., Ignatiev, K., & Grafov, D. (2020). Experience of Developing a Mobile Application Using Flutter. En K. J. Kim & H.-Y. Kim (Eds.), *Information Science and Applications* (pp. 571-575, Vol. 621). Springer. https://doi.org/10.1007/978-981-15-1465-4_56

Mahendra, M. R., & Anggorojati, B. (2020). Evaluating the Performance of Android Based Cross-Platform App Development Frameworks. *Proceedings of the 6th International Conference on Communication and Information Processing*, 32-37. https://doi.org/10.1145/3442555.3442561

Nawrocki, P., Wrona, K., Marczak, M., & Śnieżyński, B. (2021). A Comparison of Native and Cross-Platform Frameworks for Mobile Applications. *Computer*, *54*(3), 18-27. https://doi.org/10.1109/MC.2020.2983893

Suri, B., Taneja, S., Bhanot, I., Sharma, H., & Raj, A. (2022). Cross-Platform Empirical Analysis of Mobile Application Development Frameworks: Kotlin, React Native and Flutter. *Proceedings of the 4th International Conference on Information Management & Machine Intelligence*, 1-6. https://doi.org/10.1145/3590837.3590897